```java
package BINARY_SEARCH.BS;
import java.util.*;
public class microsoft_oa {

    /*


    MICROSOFT OA | 51 LPA | BINARY SEARCH ON ANSWER

    A foundry produces an alloy by combining n different metals in fixed proportions. To make exactly one unit of the alloy, you must use:
    - composition[i] units of metal i, for i = 0, 1, …, n-1.
    The foundry already has stock[i] units of each metal i on hand. If more of metal i is needed, they can buy any amount from a supplier (who has infinite sup
    at a price of cost[i] per unit. They have a total budget of budget to spend on these purchases.
    Write a function that determines the maximum number of whole alloy units the foundry can produce by using the metals in stock plus whatever additional meta
    An integer array composition of length n, where composition[i] is the units of metal i needed per alloy unit.
    An integer array stock of length n, where stock[i] is how many units of metal i are already available.
    An integer array cost of length n, where cost[i] is the price per unit of metal i.
    An integer budget, the total money available for purchases.
    Return the maximum number of alloy units that can be produced.


    Constraints n<=1e5
    budget <= 1e9

    Testcases -
5
2 3
1 2
0 1
1 1
4 30
2 2 3 1
3 2 1 4
2 3 1 6
3 0
5 5 5
5 5 5
10 10 10
3 100
4 1 2
0 0 0
10 10 10
1 10
100
0
5


Expected Output

1
3
1
1
2

     */
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        int t = s.nextInt();
        while(t-- >0) {
            int n = s.nextInt();
            int budget = s.nextInt();
            int [] req = new int [n], stock = new int [n] , cost = new int [n];

            for( int i =0 ; i < n ; i++ ) req[i]= s.nextInt();
            for( int i =0 ; i < n ; i++ ) stock[i]= s.nextInt();
            for( int i =0 ; i < n ; i++ ) cost[i]= s.nextInt();
            solve( n  ,req , stock , cost , budget);
        }

    }

    static void solve( int n, int [] req ,int [] stock , int [] cost , int  budget) {
        long lo = 0 , hi = (int)1e9 , ans = 0;
        while(lo<=hi) {
            long mid = (lo+hi) >>>1 ;
            if(check(n  ,req , stock , cost , budget , mid)) {
                ans = mid;
                lo = mid+1;

            }
            else {
                hi = mid -1;
            }
        }

        System.out.println(ans);
    }

    static boolean check (int n, int [] req ,int [] stock , int [] cost , int  budget , long mid ) {
        long total_cost = 0;
        //mid = units
        for( int i = 0; i < n ; i++  ) {
            long need = (long) mid*req[i] - stock[i];
            total_cost += need < 0 ? 0 : (long) cost[i] * need;
            if(total_cost > budget) return false;
        }
        return total_cost <= budget;
    }
}
```